Financial Portfolio Management System

A Project Report

Submitted by

KONALA MANASA- AP24110010943

Under the Supervision of

Dr. Deepthi K C Kakumani
Assistant Professor
Department of Computer Science and Engineering
SRM University-AP

In partial fulfilment for the requirements of the DBMS (CSE209) Project

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SRM UNIVERSITY-AP

NEERUKONDA

MANGALAGIRI - 522503

ANDHRA PRADESH, INDIA

APRIL-2026

CERTIFICATE

This is to certify that the project work entitled “Financial Portfolio Management System” is a Bonafide record of project work carried out by the following student:

- **Ms. Manasa Konala – AP24110010943**

as a required academic component of the course CSE 209: Data Base Management Systems during

Semester 4 of Academic Year 2025-26 in the Department of Computer Science and Engineering at SRM University, AP. This work executed under the guidance of the undersigned and is deemed to have successfully met all course requirements as of

Station: Mangalagiri

Date:22/04/2026

Assistant Professor

(Signature of the Course Instructor)

Dr. Deepthi K C Kakumani

Assistant Professor,

Department of Computer Science and Engineering,SRM University, AP

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to everyone who supported and encouraged me during the completion of this project.

I sincerely thank **Prof. Ch Satish Kumar**, Vice Chancellor, and **Professor C.V. Tomy**, Dean of the School of Engineering and Sciences (SEAS), **SRM University-AP**, for providing an excellent academic environment that promotes learning and innovation.

I am especially grateful to my faculty mentor, **Dr. Deepthi K C Kakumani**, for her constant guidance, valuable suggestions, and encouragement throughout the development of this project. Her expertise in **Database Management Systems** and continuous support played an important role in the successful completion of this work.

I would also like to thank my family and friends for their motivation and support throughout the project.

TABLE OF CONTENTS

1. Identification of Project

Project Title: Financial Portfolio Management System

This project is developed as part of the Database Management System (DBMS) course to demonstrate the practical implementation of database concepts in a real-world financial application.

The system is designed to manage user investments, track financial assets, and analyze portfolio performance using a structured relational database.

2. Project Background

In today's financial environment, individuals invest in multiple financial instruments such as stocks, bonds, mutual funds, commodities, and real estate. Managing these investments manually becomes complex due to constantly changing prices, transaction history, and risk factors.

Traditional methods like spreadsheets have limitations such as:

- Lack of automation
- Poor data integrity
- Difficulty in handling large datasets
- No relationship management between financial entities

To overcome these limitations, a **Financial Portfolio Management System** is required. This system ensures:

- Efficient storage of financial data
- Proper relationships between entities using relational database concepts
- Accurate tracking of investments and returns
- Easier analysis of portfolio performance

3. Description of the Project

The Financial Portfolio Management System is a database-driven application that helps users manage their investments effectively.

The system ensures scalability, data consistency, and efficient querying through structured relational design.

Main Objectives

- To maintain user investment records
- To track different financial assets
- To calculate profit, loss, and returns
- To categorize assets based on risk levels
- To maintain historical price data for analysis

Core Functionalities

- **User Management:** Stores user account details
- **Asset Management:** Stores financial instruments like stocks and bonds
- **Portfolio Tracking:** Tracks user investments (quantity, buy price, date)
- **Transaction Management:** Records BUY and SELL operations automatically
- **Broker Management:** Stores broker information
- **Price Tracking:** Maintains historical asset prices
- **Performance Analysis:** Calculates investment value, profit, and returns using SQL views

This system integrates relational database design with analytical queries to provide meaningful financial insights.

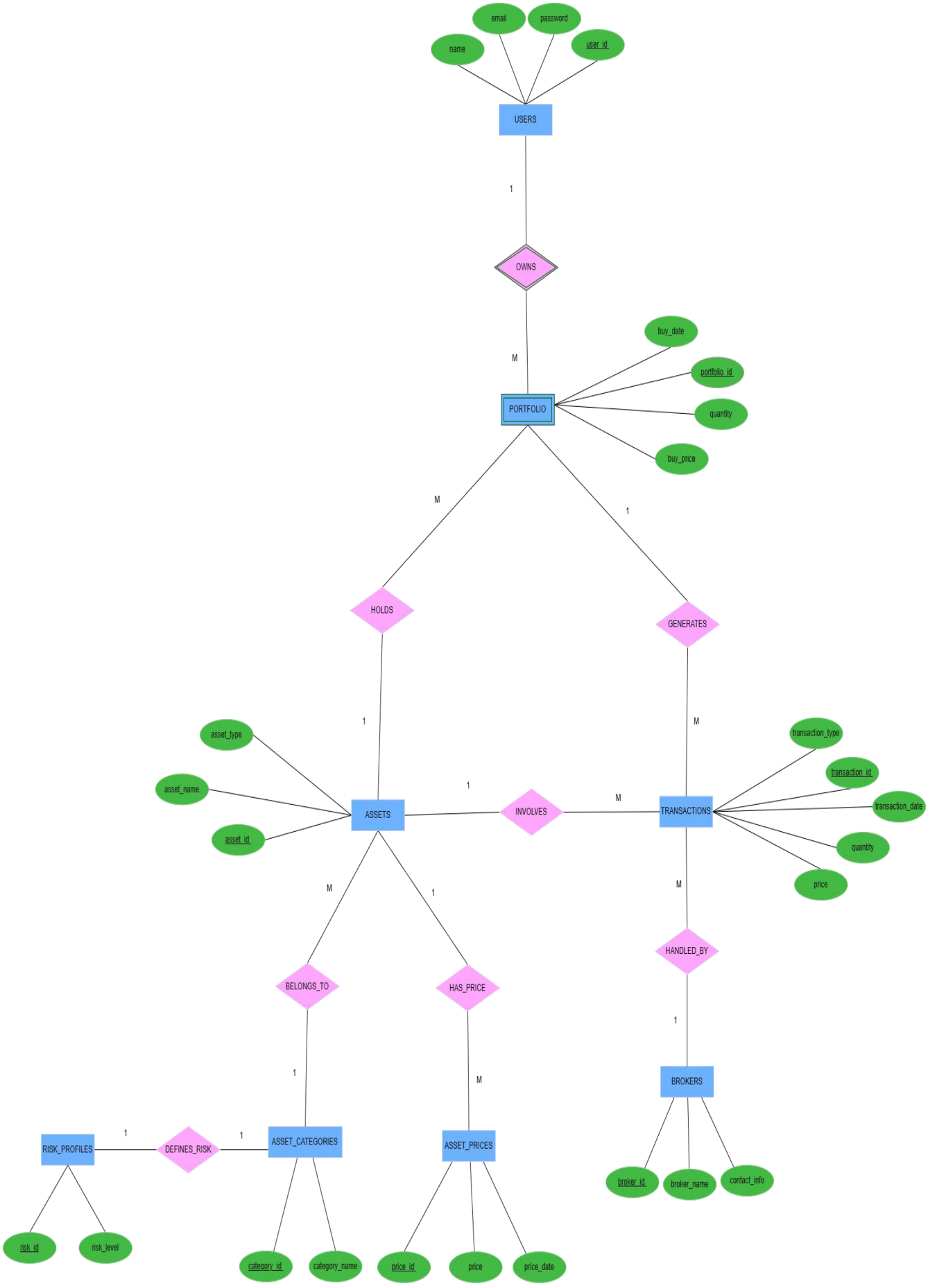
4.Entity-Relationship (ER) Diagram

The Entity-Relationship (ER) diagram is used to visually represent the structure of the database and the relationships between different entities in the system.

It is designed using **Chen's Notation**, which includes:

- **Entities** (rectangles): Users, Assets, Portfolio, etc.
- **Attributes** (ovals): name, price, quantity, etc.
- **Relationships** (diamonds): owns, invests, transacts
- **Cardinality**: 1:1, 1:M relationships

6The ER diagram clearly shows how users invest in assets, how transactions are recorded, and how asset price change over time.


email
password
name
user_id
USERS
1
OWNS
M
buy_date
portfolio_id
quantity
buy_price
PORTFOLIO
M
1
HOLDS
GENERATES
1
M
asset_type
asset_name
asset_id
ASSETS
1
INVOLVES
M
TRANSACTIONS
transaction_type
transaction_id
transaction_date
quantity
price
M
1
M
BELONGS_TO
HAS_PRICE
HANDLED_BY
1
M
1
BROKERS
RISK_PROFILES
1
DEFINES_RISK
1
ASSET_CATEGORIES
ASSET_PRICES
broker_id
broker_name
contact_info
risk_id
risk_level
category_id
category_name
price_id
price
price_date

5. Description of ER Diagram

The ER diagram consists of multiple entities and relationships that together represent the financial portfolio system.

The ER diagram consists of **8 main entities**:

Entities

- Users
- Asset Categories
- Assets
- Risk Profiles
- Portfolio
- Transactions
- Brokers
- Asset Prices

Relationships

- **Users → Portfolio (1:M):**
  One user can have multiple portfolio entries.
- **Assets → Portfolio (1:M):**
  Each asset can appear in multiple portfolios.
- **Asset Categories → Assets (1:M):**
  Each category contains multiple assets.
- **Asset Categories → Risk Profiles (1:1):**
  Each category has one associated risk level.
- **Portfolio → Transactions (1:M):**
  Each portfolio entry can generate multiple transactions.
- **Transactions → Assets (M:1):** Each transaction involves one asset, and an asset can have multiple transactions.
- **Brokers → Transactions (1:M):**
  One broker can handle many transactions.
- **Assets → Asset Prices (1:M):**
  Each asset has multiple historical price records.

Key Observations

- Ensures proper normalization (up to 3NF)
- Eliminates redundancy
- Maintains strong referential integrity
- Supports real-world financial modeling

NOTE:The **PORTFOLIO** entity is modeled as a **weak (associative) entity** because it depends on the USERS entity and represents the relationship between USERS and ASSETS. It does not exist independently and contains attributes such as quantity, buy_price, and buy_date.

The relationship **OWNS** is an **identifying relationship**, as it uniquely identifies the weak entity PORTFOLIO with the help of the primary key of USERS.

6. Conversion of ER Diagram into Tables

To translate the conceptual ER model into a structured relational database schema, all entities are converted into tables, attributes are mapped as columns, and relationships are implemented using **Primary Keys (PK)** and **Foreign Keys (FK)** to maintain referential integrity.

The design ensures proper normalization, avoids redundancy, and supports efficient querying.

The ER model is converted into relational tables where:

- **Entities** become tables
- **Attributes** become columns
- **Relationships** are implemented using **Primary Keys (PK)** and **Foreign Keys (FK)**

Based on the ER diagram, the tables in the **Financial Portfolio Management System** are classified as follows:

Independent Tables (No Foreign Keys Required)

These tables store the primary information and do not depend on other tables.

- **USERS**
- **ASSET_CATEGORIES**
- **BROKERS**

Dependent Tables (Foreign Keys Required)

These tables depend on other tables through foreign key relationships.

- **ASSETS** → references **ASSET_CATEGORIES**
- **RISK_PROFILES** → references **ASSET_CATEGORIES**
- **PORTFOLIO** → references **USERS** and **ASSETS**
- **TRANSACTIONS** → references **PORTFOLIO** and **BROKERS**
- **ASSET_PRICES** → references **ASSETS**

This classification ensures that foundational data is stored independently, while transactional and analytical data maintains proper relationships through foreign key constraints.

Relational Schema

The ER diagram is converted into the following relational schema:

USERS(user_id, name, email, password, created_at)

ASSET_CATEGORIES(category_id, category_name)

BROKERS(broker_id, broker_name, contact_info)

ASSETS(asset_id, asset_name, asset_type, category_id)

RISK_PROFILES(risk_id, category_id, risk_level)

PORTFOLIO(portfolio_id, user_id, asset_id, quantity, buy_price, buy_date)

TRANSACTIONS(transaction_id, portfolio_id, transaction_type, quantity, price, transaction_date, broker_id)

ASSET_PRICES(price_id, asset_id, price, price_date)

This relational schema represents the database structure of the **Financial Portfolio Management System**, ensuring proper linkage between users, assets, transactions, brokers, and risk profiles while maintaining data consistency.

7.Description of Tables (Database Schema)

7.1 USERS

Stores information related to system users who manage financial portfolios.

- user_id (PK): INT, Auto-increment, uniquely identifies each user
- name: VARCHAR(100), Full name of the user
- email: VARCHAR(100), Unique email address used for authentication
- password: VARCHAR(255), Encrypted user password
- created_at: TIMESTAMP, Records the account creation time

7.2 ASSET_CATEGORIES

Defines different categories of financial assets for classification and analysis.

- category_id (PK): INT, Auto-increment, unique identifier
- category_name: VARCHAR(50), Name of category (e.g., Stock, Bond, Mutual Fund)

7.3 ASSETS

Stores detailed information about individual financial instruments.

- asset_id (PK): INT, Auto-increment
- asset_name: VARCHAR(100), Name of the asset (e.g., TCS, Infosys)
- asset_type: VARCHAR(50), Type of asset (e.g., Equity, ETF, Bond)
- category_id (FK): INT, References ASSET_CATEGORIES(category_id)

7.4 RISK_PROFILES

Defines the risk level associated with each asset category.

- risk_id (PK): INT, Auto-increment
- category_id (FK): INT, References ASSET_CATEGORIES(category_id)
- risk_level: ENUM('Low', 'Medium', 'High'), Defines risk level of category
-

7.5 PORTFOLIO

Represents the investments made by users and acts as the central table in the system.

- portfolio_id (PK): INT, Auto-increment
- user_id (FK): INT, References USERS(user_id)
- asset_id (FK): INT, References ASSETS(asset_id)
- quantity: INT, Number of units purchased (must be > 0)
- buy_price: DECIMAL(12,2), Price at which asset was bought
- buy_date: DATE, Date of investment

7.6 BROKERS

Stores information about brokers who facilitate financial transactions.

- broker_id (PK): INT, Auto-increment
- broker_name: VARCHAR(100), Name of the broker (e.g., Zerodha, ICICI Direct)
- contact_info: VARCHAR(100), Contact details of the broker

7.7 TRANSACTIONS

Records all financial transactions (BUY/SELL) performed within the portfolio.

- transaction_id (PK): INT, Auto-increment
- portfolio_id (FK): INT, References PORTFOLIO(portfolio_id)
- asset_id (FK): INT, References ASSETS(asset_id)
- transaction_type: ENUM('BUY', 'SELL'), Type of transaction
- quantity: INT, Number of units transacted
- price: DECIMAL(12,2), Price at which transaction occurred
- transaction_date: DATE, Date of transaction
- broker_id (FK): INT, References BROKERS(broker_id)

7.8 ASSET_PRICES

Maintains historical price data for assets to enable trend analysis and performance evaluation.

- price_id (PK): INT, Auto-increment
- asset_id (FK): INT, References ASSETS(asset_id)
- price: DECIMAL(12,2), Market price of the asset
- price_date: DATE, Date of recorded price

8. Normalization of Tables up to 3-NF

Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. In the **Financial Portfolio Management System**, the database tables are normalized up to **Third Normal Form (3NF)** to ensure efficient data storage and consistency.

First Normal Form (1NF)

A table is said to be in **First Normal Form (1NF)** if:

- all attributes contain atomic values,
- each record is unique,
- there are no repeating groups or multivalued attributes.

In this project, all the tables satisfy **1NF** because every attribute stores only a single value, and each table has a primary key to uniquely identify each record.

For example, in the **USERS** table, the attributes **user_id**, **name**, **email**, and **password** store single values, ensuring atomicity.

Second Normal Form (2NF)

A table is in **Second Normal Form (2NF)** if:

- it is already in 1NF,
- all non-key attributes are fully dependent on the primary key.

In the Financial Portfolio Management System, all tables satisfy **2NF** because each table has a single-column primary key, and all non-key attributes depend completely on that primary key.

For example, in the **PORTFOLIO** table, the attributes **quantity**, **buy_price**, and **buy_date** depend entirely on **portfolio_id**. Thus, there are no partial dependencies in the database.

Third Normal Form (3NF)

A table is in **Third Normal Form (3NF)** if:

- it is in 2NF,

- there are no transitive dependencies,
- non-key attributes do not depend on other non-key attributes.

In this project, the database schema satisfies **3NF** by separating related information into different tables.

For example:

- **ASSET_CATEGORIES** stores category details,
- **RISK_PROFILES** stores risk levels,
- **BROKERS** stores broker details.

This separation ensures that non-key attributes depend only on the primary key and not on other non-key attributes.

For instance, the **risk_level** attribute is stored in the **RISK_PROFILES** table rather than in the **ASSETS** table, thereby eliminating transitive dependency.

Hence, the database design of the Financial Portfolio Management System is normalized up to **Third Normal Form (3NF)**, which minimizes redundancy and improves consistency.

9. Creation of Tables

The following SQL statements are used to create the tables required for the **Financial Portfolio Management System**. These tables store user details, portfolio information, asset data, broker details, transactions, asset prices, and risk profiles.

Creating USERS Table

The **USERS** table stores the details of the users who maintain investment portfolios.

```
CREATE TABLE users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Creating ASSET_CATEGORIES Table

The **ASSET_CATEGORIES** table stores the different categories of assets such as stocks, bonds, and mutual funds.

```
CREATE TABLE asset_categories (
    category_id INT AUTO_INCREMENT PRIMARY KEY,
    category_name VARCHAR(50) UNIQUE NOT NULL
);
```

Creating ASSETS Table

The **ASSETS** table stores the investment asset details.

```
CREATE TABLE assets (
    asset_id INT AUTO_INCREMENT PRIMARY KEY,
    asset_name VARCHAR(100) NOT NULL,
    asset_type VARCHAR(50) NOT NULL,
    category_id INT NOT NULL,
    FOREIGN KEY (category_id)
        REFERENCES asset_categories(category_id)
        ON DELETE CASCADE
);
```

Creating RISK_PROFILES Table

The **RISK_PROFILES** table stores the risk level associated with each asset category.

```
CREATE TABLE risk_profiles (
    risk_id INT AUTO_INCREMENT PRIMARY KEY,
    category_id INT NOT NULL,
    risk_level ENUM('Low','Medium','High') NOT NULL,
    FOREIGN KEY (category_id)
        REFERENCES asset_categories(category_id)
        ON DELETE CASCADE
);
```

Creating PORTFOLIO Table

The **PORTFOLIO** table stores the investment records of users.

```
CREATE TABLE portfolio (
    portfolio_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    asset_id INT NOT NULL,
    quantity INT NOT NULL CHECK (quantity > 0),
    buy_price DECIMAL(12,2) NOT NULL,
    buy_date DATE NOT NULL,
    FOREIGN KEY (user_id)
        REFERENCES users(user_id)
        ON DELETE CASCADE,
    FOREIGN KEY (asset_id)
        REFERENCES assets(asset_id)
        ON DELETE CASCADE
);
```

Creating BROKERS Table

The **BROKERS** table stores the broker information involved in transactions.

```
CREATE TABLE brokers (
    broker_id INT AUTO_INCREMENT PRIMARY KEY,
    broker_name VARCHAR(100) NOT NULL,
    contact_info VARCHAR(100)
);
```

Creating TRANSACTIONS Table

The **TRANSACTIONS** table stores the details of asset transactions made by users.

```
CREATE TABLE transactions (
    transaction_id INT AUTO_INCREMENT PRIMARY KEY,
    portfolio_id INT NOT NULL,
    asset_id INT NOT NULL,
    transaction_type ENUM('BUY','SELL') NOT NULL,
    quantity INT NOT NULL,
    price DECIMAL(12,2) NOT NULL,
    transaction_date DATE NOT NULL,
    broker_id INT,

    FOREIGN KEY (portfolio_id)
        REFERENCES portfolio(portfolio_id)
        ON DELETE CASCADE,

    FOREIGN KEY (asset_id)
        REFERENCES assets(asset_id)
        ON DELETE CASCADE,

    FOREIGN KEY (broker_id)
        REFERENCES brokers(broker_id)
        ON DELETE SET NULL
);
```

Creating ASSET_PRICES Table

The **ASSET_PRICES** table stores the historical prices of assets for trend analysis.

```
CREATE TABLE asset_prices (
    price_id INT AUTO_INCREMENT PRIMARY KEY,
    asset_id INT NOT NULL,
    price DECIMAL(12,2) NOT NULL,
    price_date DATE NOT NULL,
    FOREIGN KEY (asset_id)
        REFERENCES assets(asset_id)
        ON DELETE CASCADE
);
```

10. Creation of Data in the Tables

After creating the required tables, sample records are inserted into the tables to test the functionality of the **Financial Portfolio Management System**. These records represent users, asset categories, assets, portfolios, brokers, transactions, and historical asset prices

The inserted sample data is used to test portfolio tracking, risk analysis, transaction management, and performance reporting features of the **Financial Portfolio Management System**.

```sql
INSERT INTO users (name, email, password)
VALUES ('Manasa', 'manasa@gmail.com', '1234');

INSERT INTO asset_categories (category_name) VALUES
('Stock'), ('Bond'), ('Real Estate'), ('Mutual Fund'), ('Commodity');

INSERT INTO risk_profiles (category_id, risk_level) VALUES
(1,'High'), (2,'Low'), (3,'Medium'), (4,'Medium'), (5,'High');

INSERT INTO assets (asset_name, asset_type, category_id) VALUES
('TCS','Equity',1),
('Infosys','Equity',1),
('HDFC Bond','Corporate Bond',2),
('Govt Bond','Government Bond',2),
('DLF Ltd','Real Estate',3),
('SBI Mutual Fund','Open-Ended Fund',4),
('Gold ETF','Commodity ETF',5);

INSERT INTO brokers (broker_name, contact_info) VALUES
('Zerodha','contact@zerodha.com'),
('ICICI Direct','support@icicidirect.com');

-- PORTFOLIO DATA (TRIGGER AUTO POPULATES TRANSACTIONS)

INSERT INTO portfolio (user_id, asset_id, quantity, buy_price, buy_date) VALUES
(1,1,10,3500,'2024-01-10'),
(1,2,20,3000,'2024-02-15'),
(1,3,50,1000,'2024-03-10'),
(1,4,30,1200,'2024-04-20'),
(1,5,2,55000,'2024-05-01'),
(1,6,100,500,'2024-06-01'),
(1,7,4,2500,'2024-07-01');

INSERT INTO asset_prices (asset_id, price, price_date) VALUES
(1,4100,'2024-07-01'),
(2,3500,'2024-07-01'),
(3,1100,'2024-07-01'),
(4,1250,'2024-07-01'),
(5,58000,'2024-07-01'),
(6,620,'2024-07-01'),
(7,2700,'2024-07-01');
```

11. Write SQL Queries (Subqueries, Aggregate Functions, Joins)

The following SQL queries are used to retrieve meaningful information from the **Financial Portfolio Management System** database. These queries demonstrate the use of **joins**, **aggregate functions**, and **subqueries** for portfolio analysis and reporting.

Query 1: Join Query – Displaying Portfolio Details

This query retrieves the details of users along with the assets they own, quantity purchased, and purchase price by joining the **USERS**, **PORTFOLIO**, and **ASSETS** tables.

```sql
SELECT u.name, a.asset_name, p.quantity, p.buy_price
FROM PORTFOLIO p
JOIN USERS u ON p.user_id = u.user_id
JOIN ASSETS a ON p.asset_id = a.asset_id;
```

This query helps in viewing the investment details of each user.

Query 2: Aggregate Function – Calculating Total Investment

This query calculates the total amount invested by each user using the **SUM()** aggregate function.

```
SELECT user_id, SUM(quantity * buy_price) AS total_investment
FROM PORTFOLIO
GROUP BY user_id;
```

This query helps in calculating the total investment made by every user.

Query 3: Subquery – Users with Above Average Investment

This query identifies users whose total investment is greater than the average investment of all users.

```
SELECT name
FROM USERS
WHERE user_id IN (
    SELECT user_id
    FROM PORTFOLIO
    GROUP BY user_id
    HAVING SUM(quantity * buy_price) >
    (SELECT AVG(quantity * buy_price) FROM PORTFOLIO)
);
```

This query helps in identifying high-investment users.

Query 4: Join Query – Displaying Asset Risk Levels

This query displays each asset along with its corresponding risk level by joining the **ASSETS**, **ASSET_CATEGORIES**, and **RISK_PROFILES** tables.

```
SELECT a.asset_name, r.risk_level
FROM ASSETS a
JOIN ASSET_CATEGORIES ac ON a.category_id = ac.category_id
JOIN RISK_PROFILES r ON ac.category_id = r.category_id;
```

This query helps in analyzing the risk associated with each asset.

Query 5: Aggregate Query – Number of Assets per User

This query counts the number of assets owned by each user.

```
SELECT user_id, COUNT(asset_id) AS total_assets
FROM PORTFOLIO
GROUP BY user_id;
```

This query helps in determining the number of investments held by each user.

The above queries help in portfolio monitoring, investment analysis, and risk assessment in the **Financial Portfolio Management System**.

12. Creation of Views Using the Tables

Views are virtual tables created using SQL queries to simplify data retrieval and reporting. In the **Financial Portfolio Management System**, views are created to summarize portfolio performance and analyze asset risks efficiently.

View 1: Portfolio Summary View

This view displays the total investment made by each user by joining the **USERS** and **PORTFOLIO** tables.

```
CREATE VIEW portfolio_summary_view AS
SELECT
    u.name,
    SUM(p.quantity * p.buy_price) AS total_investment
FROM USERS u
JOIN PORTFOLIO p ON u.user_id = p.user_id
GROUP BY u.name;
```

This view helps in summarizing the investment amount for each user.

View 2: Asset Risk View

This view displays the assets along with their associated risk levels by joining the **ASSETS**, **ASSET_CATEGORIES**, and **RISK_PROFILES** tables.

```
CREATE VIEW asset_risk_view AS
SELECT
    a.asset_name,
    r.risk_level
FROM ASSETS a
JOIN ASSET_CATEGORIES ac ON a.category_id = ac.category_id
JOIN RISK_PROFILES r ON ac.category_id = r.category_id;
```

This view helps in identifying the risk category of each asset.

These views simplify complex queries and provide summarized information for performance tracking and risk analysis in the **Financial Portfolio Management System**.

```sql
CREATE VIEW portfolio_summary_view AS
SELECT
    u.user_id,
    u.name AS user_name,
    a.asset_name,
    a.asset_type,
    c.category_name,
    rp.risk_level,
    p.quantity,
    p.buy_price,
    ap.price AS current_price,
    (p.quantity * p.buy_price) AS investment,
    (p.quantity * ap.price) AS current_value,
    ((ap.price - p.buy_price) * p.quantity) AS profit,
    ROUND(((ap.price - p.buy_price) / p.buy_price) * 100, 2) AS return_percent
FROM portfolio p
JOIN users u ON p.user_id = u.user_id
JOIN assets a ON p.asset_id = a.asset_id
JOIN asset_categories c ON a.category_id = c.category_id
LEFT JOIN risk_profiles rp ON c.category_id = rp.category_id
JOIN asset_prices ap ON a.asset_id = ap.asset_id
WHERE ap.price_date = (
    SELECT MAX(price_date)
    FROM asset_prices
    WHERE asset_id = a.asset_id
);
```

13. Conclusion

The **Financial Portfolio Management System** was successfully designed and implemented using **SQL** for efficient portfolio tracking and investment management. The project enables users to manage their investments in various asset categories such as stocks, bonds, and mutual funds.

The database design was developed using proper **ER modeling**, relational table creation, and normalization up to **Third Normal Form (3NF)** to ensure data integrity and reduce redundancy. SQL queries, joins, aggregate functions, and views were implemented to support portfolio analysis, risk evaluation, and reporting.

This project demonstrates how database management concepts can be applied to create an effective system for managing financial investments and generating meaningful insights from portfolio data.

14. Future Scope

The **Financial Portfolio Management System** can be further enhanced by integrating **Python** for advanced financial analysis, automated report generation, and graphical data visualization.

Future improvements may include:

- Integration with real-time stock market APIs for live asset price updates
- Dashboard visualization for portfolio performance tracking
- Automated profit/loss analysis reports
- User authentication with enhanced security features
- Mobile or web-based application interface for easy access

These enhancements would improve the usability and analytical capabilities of the system, making it more efficient and user-friendly.

15. References

1. **Elmasri, R. and Navathe, S. B.**, *Fundamentals of Database Systems*, Pearson Education.
2. **Oracle MySQL Documentation**, "MySQL Reference Manual." Available at: https://dev.mysql.com/doc/
3. **Lucidchart / Draw.io**, Online ER Diagram Tool. Available at: https://app.diagrams.net/
4. **W3Schools SQL Tutorial**, Available at: https://www.w3schools.com/sql/
5. Course materials and lecture notes provided for **Database Management Systems**, **SRM University-AP**.